



nextwork.org

Set Up a RAG Chatbot in Bedrock



Natasha Ong

<https://www.linkedin.com/in/natasha-ong/>



What is NextWork



NextWork is an organization that provides projects and resources for learning and development, with the goal of helping people find jobs they love.^[1]

[Show details >](#)



Introducing Today's Project!

RAG (Retrieval Augmented Generation) is an AI technique that lets you train an AI model on your own personal documents. In this project, we will demonstrate RAG by setting up a RAG chatbot in Amazon Bedrock

Tools and concepts

Services we used in this project were Amazon Bedrock, S3, and OpenSearch Serverless. Key concepts we learnt include knowledge bases, requesting access to AI models, how chatbots generate responses (i.e. AI models + knowledge base), and vector stores.

Project reflection

This project took us approximately two hours including project demo time. The most challenging part was the error with our AI models (and understanding on-demand vs pre-provisioned inference). It was most rewarding to level up our chatbot's responses

We did this project today to learn more about Bedrock and RAG. This project definitely met our goals - awesome to learn both over a hands on project!



Understanding Amazon Bedrock

Amazon Bedrock is an AWS service that makes it easy to build generative AI applications - it's like an AI model marketplace that lets us find, use and test models from different providers. We're using Bedrock to create a Knowledge Base.

Our Knowledge Base is connected to S3 because S3 is going to be the storage/source for our Knowledge Base's raw documents. S3 is AWS's storage service, where you can store all kinds of objects (e.g. videos, documents, audio) in the same bucket.

In an S3 bucket, we uploaded the documents that will make up our chatbot's knowledge. Our S3 bucket is in the same region as our Knowledge Base because Bedrock is a regional service - data must live in the same region as the Bedrock resource (Kbase).

Files and folders

Configuration

Files and folders (10 total, 138.3 MB)

Find by name

Name	Folder	Type	Size	Status
Prompt Engineering.pdf	-	application/pdf	16.4 MB	Pending
Create S3 Buckets with Terraform.pdf	-	application/pdf	16.5 MB	Pending
Build a Three-Tier Web App.pdf	-	application/pdf	16.6 MB	Pending
Building an AI Workflow.pdf	-	application/pdf	16.4 MB	Succeeded
Fetch Data with AWS Lambda.pdf	-	application/pdf	16.0 MB	Succeeded
Deploy Backend with Kubernetes.pdf	-	application/pdf	15.3 MB	Succeeded
Transcribe Audio Files with AI.pdf	-	application/pdf	13.7 MB	Succeeded
How to Use DeekSeek.pdf	-	application/pdf	6.2 MB	Succeeded



My Knowledge Base Setup

Our Knowledge Base uses a vector store, which means a search engine/database that stores data based on their semantic meaning! When we query our Knowledge Base, OpenSearch will find the relevant chunks of data to the query, and passed it to Bedrock.

Embeddings are vector representations of the semantic meaning of a chunk of text. The embedding model we're using is Titan Text Embeddings v2 because it's fast, accurate and a lot more affordable!

Chunking is the process of splitting up text into smaller pieces i.e. chunks. This helps with searching for data more efficiently in the vector store. In our Knowledge Base, chunks are set to be about 300 tokens in size each!

Step 1: Provide details

Knowledge Base details

Knowledge Base name network-rag-documentation	Knowledge Base description This Knowledge base stores all documentation at NextWork.	Service role AmazonBedrockExecutionRoleForKnowledgeBase_u8bely
Knowledge base type Knowledge base use vector store	Data source type S3	Log Deliveries —

Edit

Step 2: Setup up data source

Data source: s3-bucket-nextwork-rag-bedrock

Data source name s3-bucket-nextwork-rag-bedrock	Account ID 471112976395 (this account)	S3 URI s3://network-rag-bedrock
Customer-managed KMS Key for S3 —	KMS key for transient data storage —	Chunking strategy Default
Parsing strategy DEFAULT	Lambda function —	S3 bucket for Lambda function —
Data deletion policy DELETE		

Edit

Step 3: Select embeddings model and configure vector store

Embeddings model

Model Titan Text Embeddings v2	Vector dimensions 1024
-----------------------------------	---------------------------

Edit



AI Models

AI models are important for my chatbot because they're the translator of my Knowledge Base's search results into human-like text. Without AI models, our chatbot would only respond with chunks of text from our documents - not the best experience!

To access an AI model in Bedrock, we had to visit the "Model Access" page and request access explicitly! AWS needs explicit access because some AI model providers have extra forms/rules if you wanted to use them, and AWS needs to check availability.

▼ Amazon (4)	1/4 access granted
Titan Text Embeddings V2	✔ Access granted
Nova Pro Cross-region inference	⌚ Available to request
Nova Lite Cross-region inference	⌚ Available to request
Nova Micro Cross-region inference	⌚ Available to request
▶ Anthropic (4)	0/4 access granted
▼ Meta (8)	3/8 access granted
Llama 3.3 70B Instruct	✔ Access granted
Llama 3.2 1B Instruct Cross-region inference	⌚ Available to request
Llama 3.2 3B Instruct Cross-region inference	⌚ Available to request
Llama 3.2 11B Vision Instruct Cross-region inference	⌚ Available to request
Llama 3.2 90B Vision Instruct Cross-region inference	⌚ Available to request
Llama 3.1 405B Instruct Cross-region inference	⌚ Available to request
Llama 3.1 70B Instruct Cross-region inference	✔ Access granted
Llama 3.1 8B Instruct Cross-region inference	✔ Access granted



Syncing the Knowledge Base

Even though we've already connected our S3 bucket when creating the Knowledge Base, we still need to sync our data, which will actually move the data from S3 and into our Knowledge Base + OpenSearch Serverless. Currently, the Knowledge Base is empty!

The sync process involves three key steps: Ingesting (i.e. Bedrock takes the data from S3), Processing (i.e. Bedrock chunks and embeds the data) and Storing (i.e. Bedrock stores the processed data in the vector store, OpenSearch Serverless).

✓ Sync completed for data source - 's3-bucket-nextwork-rag-bedrock'

[Edit page.](#)

Retrieval-Augmented Generation (RAG) type
Vector store

Data source (1)

↻ Sync

⏹ Stop sync

Add

Add documents from S3

▼

Data sources contain information returned when querying a Knowledge Base.

🔍 Find data source

< 1 >

✓	Data source name	▼	Status	Data sour...	Accou
✓	s3-bucket-nextwork-rag-bedrock		✓ Available	S3	47111

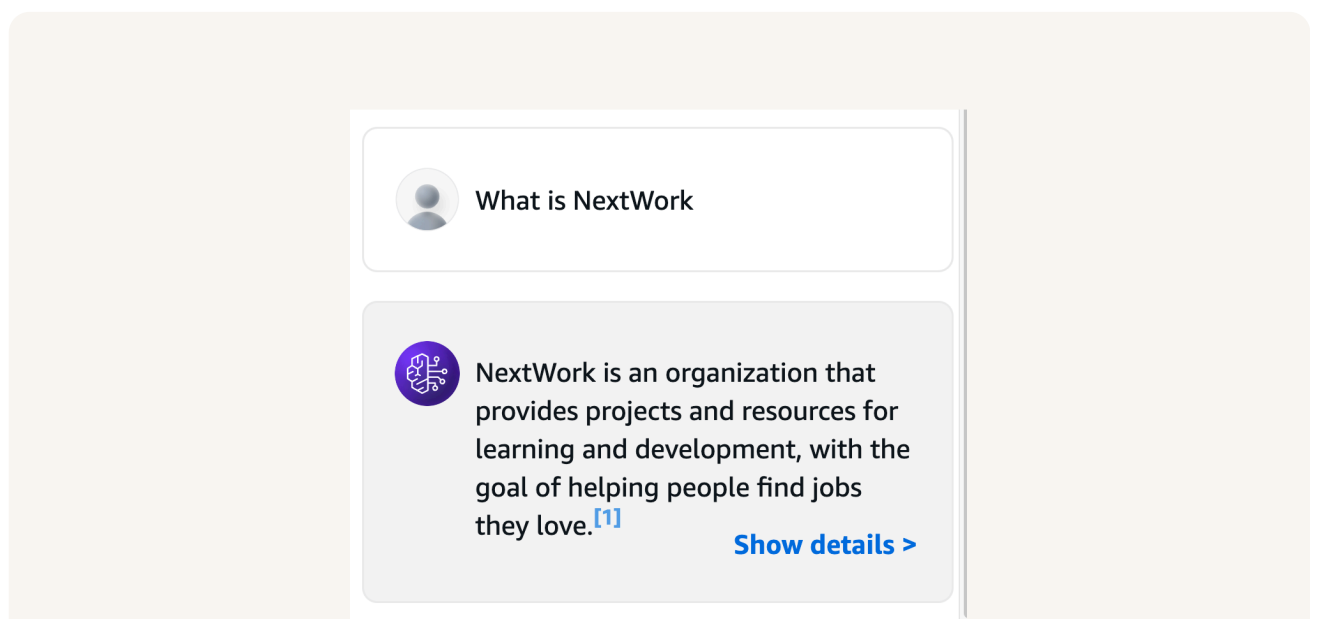


Testing My Chatbot

We initially tried to test our chatbot using Llama 3.1 8B as the AI model, but it triggered an error - it was not available on demand. We had to switch to Llama 3.3 70B because it was offered on-demand by AWS (since it's a newer, efficient model).

When we asked about topics unrelated to our Knowledge Base's data, our chatbot could not help us with the request. This proves that the chatbot only knows the information that we give it - it won't know anything that's outside of our Kbase!

You can also turn off the Generate Responses setting to see the raw chunks of data directly from our Knowledge Base. When we tested this, our chatbot gave a list of 5 paragraphs to answer a question, whereas the AI model will convert it to a sentence



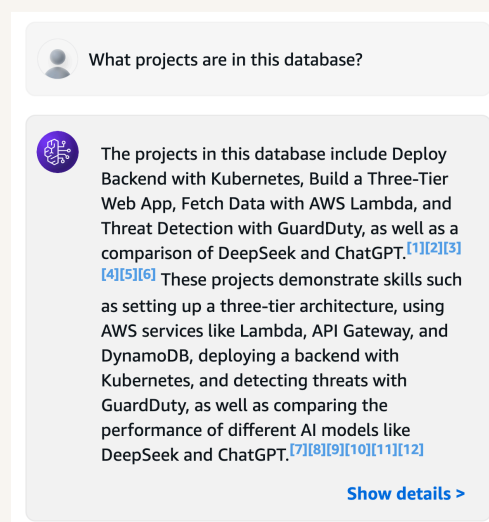


Upgrading My Chatbot

In a project extension, I looked into increasing the number of source chunks, which are the number of chunks of text that the Knowledge Base will return for a query. This will improve our chatbot's responses by giving our AI model more data to use.

We also added a custom prompt that tells our chatbot to always respond by mentioning skills that the student's learnt, even if the student didn't ask for it directly. Edit your answer

After adjusting the source chunks and the generation prompt, our chatbot's response became so much more detailed! The chatbot would talk about skills we've learnt, and used 10 or more examples within the response.





NextWork.org

Everyone should be in a job they love.

Check out nextwork.org for
more projects

